Team Contributions: Final
RoCam

Team #3, SpaceY
Zifan Si
Jianqing Liu
Mike Chen
Xiaotian Lou

March 25, 2026

This document summarizes the contributions of each team member for the final demonstration and documentation. The time period of interest is the time between Rev 0 and the Final documentation; the contributions prior to Rev0 are NOT included.

1 Team Meeting Attendance

Student	**Meetings**
Total	11
Xiaotian Lou	11
Zifan Si	11
Jianqing Liu	11
Shike Chen	11

2 Supervisor/Stakeholder Meeting Attendance

Supervisor's Name: Sirouspour, Shahin

Student	**Meetings**
Total	0
Xiaotian Lou	0
Zifan Si	0
Jianqing Liu	0
Shike Chen	0

3 Lecture Attendance

Student	Lectures
Total	1
Xiaotian Lou	0
Zifan Si	0
Jianqing Liu	0
Shike Chen	0

4 TA Document Discussion Attendance

TA's Name: Tanya Djavaherpour

Student	Lectures
Total	2
Xiaotian Lou	2
Zifan Si	2
Jianqing Liu	2
Shike Chen	2

5 Commits

Student	Commits	Percent
Total	368	100.0%
Xiaotian Lou	97	26.4%
Jianqing Liu	96	26.1%
Zifan Si	108	29.3%
Shike Chen	67	18.2%

Notes. Commit counts reflect merges to the `main` branch as well as contributions in experimental branches (including Copilot-assisted branches such as `copilot/sub-pr-302`, `copilot/sub-pr-302-again`, and `copilot/explore-repo` attributed to the team member who initiated them. During the Final period, Shike Chen focused primarily on the labelling application, SAM backend integration, and data pipeline tooling, while Xiaotian Lou concentrated on unit testing, ML model benchmarking, and TensorRT inference optimization. Much of this work involved on-device iterations, training runs, and tooling validation that lived in feature branches or local workspaces before being upstreamed. As a result, the raw `main`-branch totals alone would under-represent their effort; the adjusted figures above provide a more balanced picture.

6 Issue Tracker

Student	Authored (O+C)	Assigned (C only)
Xiaotian Lou	4	3
Zifan Si	4	5
Jianqing Liu	31	11
Shike Chen	9	8

Notes.

- **Counting rule:** "Authored (O+C)" counts issues opened by a member (open or closed). "Assigned (C only)" counts issues where the member was the assignee at closure.
- **Role-based workflow:** During the Final period, Jianqing Liu (project manager, "Pega") centrally opened, assigned, and closed most issues based on team decisions captured in meeting notes; hence his "Authored" count is higher.
- **Credit vs. logistics:** Centralized opening/closing is administrative. Actual implementation credit is reflected in commits, PRs, and code reviews; an issue can have multiple contributors beyond the final assignee.
- **Assignment semantics:** Ownership may change during an issue's lifetime; we attribute "Assigned (C only)" to the final assignee at closure to represent delivery responsibility.

7 CICD

We use GitHub Actions with five workflows that cover documentation, firmware, backend, and the operator UI.

build-tex (push to `main`, `docs/**`, or manual) Compiles LaTeX with TeX Live 2025 on Ubuntu and commits the built PDFs back to the repository. The job runs `make -B` under `docs/` and then auto-commits any updated `.pdf` files (`contents: write`).

check-tex (pull requests touching `docs/**`) Validates that the LaTeX compiles on PRs, uploads any changed PDFs as an artifact, then formats all `.tex` sources with `latexindent` (80-column wrapping) and commits the formatting changes. This keeps the documentation reproducible and consistently formatted.

firmware-ci (push/PR under `src/pcb-firmware/**`) Builds the embedded firmware with Cargo on Ubuntu and uploads the resulting binary as an artifact. The job also enforces Rust source formatting via `cargo fmt --check`.

backend-ci (pull requests touching `src/backend/**`) Runs backend lint checks on Ubuntu using Python 3.10 with pip dependency caching. The workflow installs dependencies from `src/backend/requirement.txt` (excluding any pinned `pip` line) and performs a basic syntax check via `python -m compileall -q .` in `src/backend`. This helps catch Python syntax errors early before merging.

react-ci (push/PR for the React app) Builds and smoke-tests the operator UI on Node 20. Dependencies are installed with `npm ci`; a lightweight `ci:smoke` runs on every change. Unit tests and production build currently run in *best-effort* mode (non-blocking), and any build artifacts are uploaded when present.

8 Team Charter Trigger Items

Quantified Triggers (per Team Charter)

- **Meeting cadence & attendance:** weekly team meetings; prior notice required for any absence; target attendance $\geq$85%.
- **Preparation & quality:** come to meetings with concise updates, blockers, and options; maintain decision logs and lightweight RACI notes.
- **PR/issue hygiene:** every task has a GitHub issue with an owner and estimate; all code changes go through PRs that reference an issue; no direct pushes to `main`.
- **Review SLA & CI gates:** at least one reviewer approval; address review comments within 7 days (unless a different SLA is agreed); CI must be green (docs build, firmware build, UI smoke).
- **Documentation standards:** LaTeX compiles reproducibly on CI; PDFs are published on `main`; `latexindent` enforces consistent formatting on PRs.
- **Runtime KPIs (demo readiness):** track end-to-end latency (p50/p95), FPS, target re-acquisition time, crash-free session rate, and UI task success; for the final demo we target $\sim$100–500 ms latency, $\geq$15 FPS, and $\leq$2 s re-acquisition.
- **Underperformance trigger:** if a member underdelivers for >3 weeks, initiate pairing/guardrails and, if needed, escalate to TA/instructor.

Violations Observed During the Final Period

None observed. All members met the charter triggers: weekly meetings were held with advance notice for any conflicts; PRs referenced issues and passed CI; review comments were addressed within the 7-day SLA; documentation built reproducibly; runtime KPIs were recorded for the demo.

Plan to Sustain Compliance

- Keep branch protection (no direct pushes to `main`; $\geq$1 review; CI required).
- Maintain formatting gates: `latexindent` for LaTeX, `cargo fmt` for Rust firmware, and lint checks for Python and React sources.
- Continue using meeting-issue templates (owner, ETA, blockers) and maintain the decision log to preserve cadence and traceability.
- Monitor runtime KPI thresholds (p50/p95 latency, FPS, re-acquisition time) on each demo and EXPO run.

9 Additional Productivity Metrics

At this time, we have no additional productivity metrics to report.